

AlignmentQueue.c 코드 설명

❖ 개요

이 코드는 **선형 큐(Linear Queue)**를 배열을 사용해 구현한 예제입니다. 고정된 크기의 배열을 기반으로, 큐의 기본 연산인 삽입(enqueue), 삭제(dequeue), 출력(queue_print) 등을 다룹니다.

☑ 주요 구조 및 기능 설명

▶ 객체 & 추종객을 형식체와 함께 정의

```
#define MAX_QUEUE_SIZE 5

typedef int element;
typedef struct {
    int front;
    int rear;
    element data[MAX_QUEUE_SIZE];
} QueueType;
```

- `front`와 `rear`는 큐의 시작과 끝을 가리키는 인덱스
- `data[]`는 큐에 저장된 요소 배열
- `MAX_QUEUE_SIZE`는 큐의 최대 크기 (5)

▶ 귀여하기 프로세스 (init, is_full, is_empty)

```
void init_queue(QueueType *q);
int is_full(QueueType *q);
int is_empty(QueueType *q);
```

- `init_queue`: 큐 초기화 (`front = -1, rear = -1`)
- `is_full`: `rear == MAX_QUEUE_SIZE - 1`
- `is_empty`: `front == rear`

▶ 입력 enqueue()

```
void enqueue(QueueType *q, int item);
```

- 꽉 차면 오류 출력
- 아니면 `rear` 증가 후 데이터 삽입

▶ 제거 dequeue()

```
int dequeue(QueueType *q);
```

- 비었으면 오류 출력
- 아니면 **front** 증가 후 값 반환

▶ 출력 queue_print()

```
void queue_print(QueueType *q);
```

- 배열 전체를 반복하며, 현재 큐에 해당하는 요소만 출력

학습 예시: 작동 순서

```
init_queue(&q);      // front, rear = -1
enqueue(&q, 10);     // rear = 0, data[0] = 10
enqueue(&q, 20);     // rear = 1, data[1] = 20
enqueue(&q, 30);     // rear = 2, data[2] = 30

queue_print(&q);     // 출력: 10 | 20 | 30

item = dequeue(&q);  // front = 0, 반환값 = 10
item = dequeue(&q);  // front = 1, 반환값 = 20

queue_print(&q);     // 출력:      30
```

⚠ 경고 & 패턴

- 삽입/삭제 시 **포인터 이동만 있음**, 실제 데이터 이동 없음
- 삭제된 공간을 재사용하지 못함 → **선형 큐의 단점**

평가

- 간단하고 다른 구조 및 방식을 하기 전 가장 아래된 구현
- 계획: **회전 키워드 파일** (일명: **CircleQueue.c**)으로 건너가 필요

파일 명

AlignmentQueue.c: 고정형 서비스 시뮬리서 출간 평균을 예상해 한 회전열의 구현